BACKEND ENGINEERING · HLD INTERVIEW PREP

Rate Limiting

API Throttling — Complete Study Notes

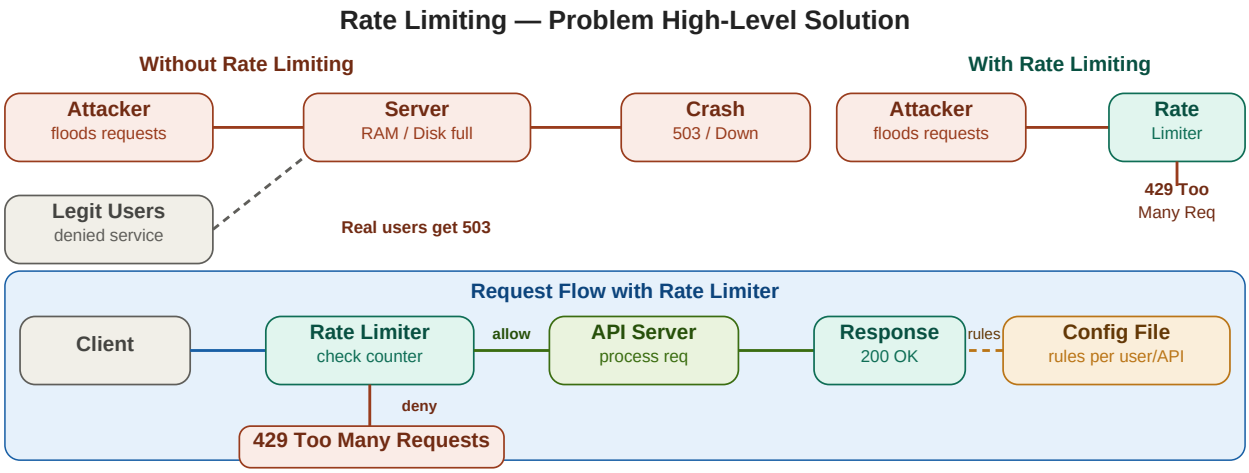
Token Bucket · Leaky Bucket · Fixed Window Counter
Sliding Window Log · Sliding Window Counter · HTTP 429

TOPICS COVERED

Table of Contents

1 & 2. The Problem & High-Level Design

Rate limiting restricts the number of API calls a user/client can make within a time window. Without it, attackers can flood your servers with requests, exhausting RAM, disk, and CPU — causing legitimate users to get 503 errors or no response at all.



Without rate limiting: attacker floods server → crash → legit users denied · With: rate limiter returns 429, server stays alive

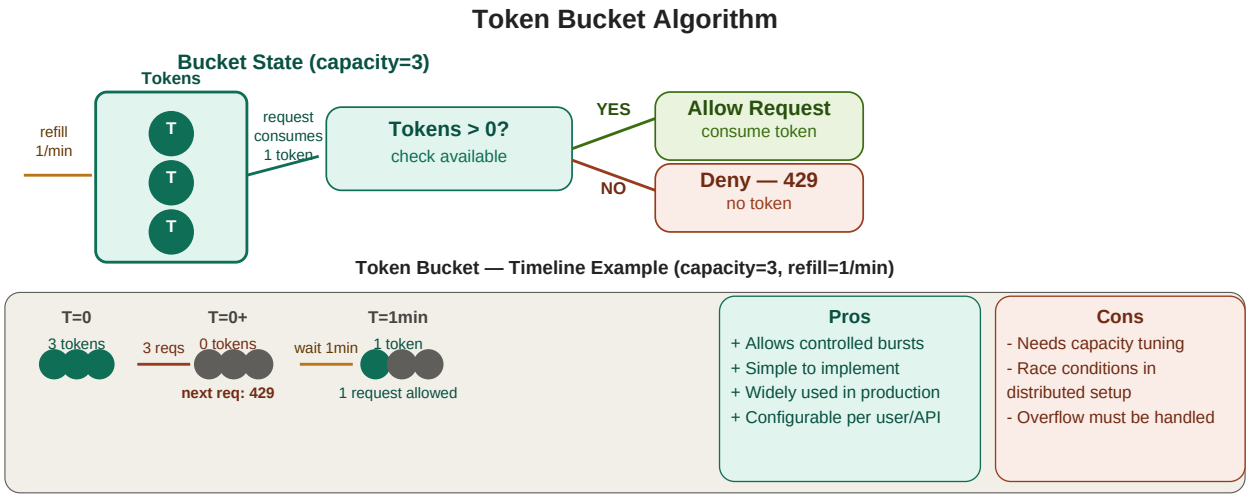
HTTP 429 Too Many Requests: the correct status code when rate limit is exceeded.

Config-driven: rules stored in config files — max requests per minute, per API, per user — cached locally for speed.

Distributed systems: centralized stores (Redis) needed to synchronize counters across multiple servers.

3. Token Bucket Algorithm

The most popular rate limiting algorithm. A bucket holds a maximum number of tokens. Tokens refill at a fixed rate. Each request consumes one token. If no tokens remain, the request is rejected with 429.



Token Bucket: capacity=3, refill=1/min · 3 requests allowed immediately, then wait for refill · controlled burst allowed

How it handles bursts

- If a user sends 0 requests for 3 minutes, they accumulate 3 tokens (up to capacity)
- They can then send 3 requests in rapid succession (burst) — all allowed
- After burst, must wait for tokens to refill before more requests
- This is different from leaky bucket which processes at constant rate regardless

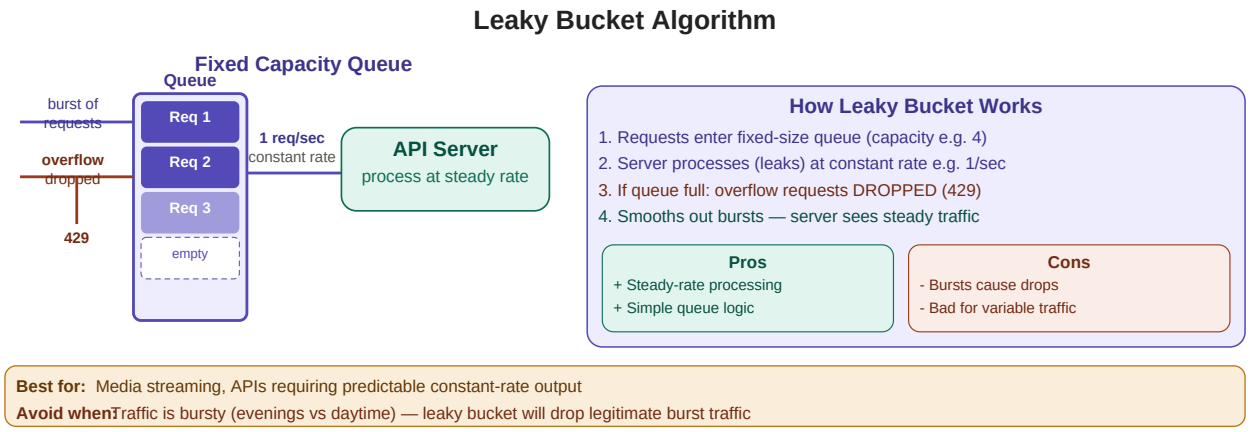
Implementation: maintain (token_count, last_refill_time) per user in Redis.

On each request: tokens += rate * (now - last_refill); tokens = min(tokens, capacity); if tokens >= 1: tokens--; allow; else: deny 429.

Used by: AWS API Gateway, Stripe, GitHub API, most major API providers.

4. Leaky Bucket Algorithm

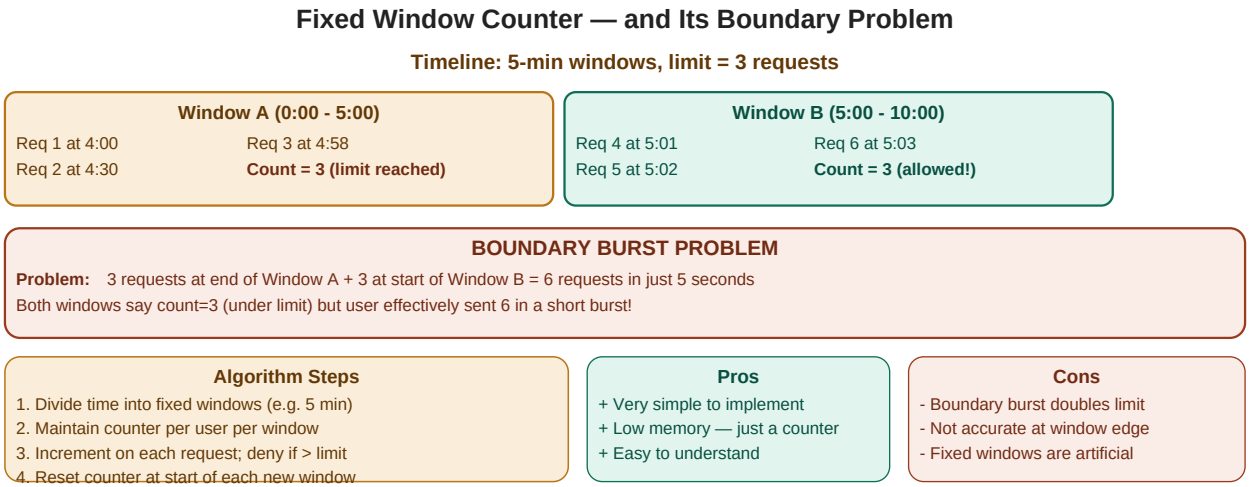
Requests enter a fixed-size queue. The queue "leaks" (processes) requests at a constant rate. If the queue is full, new requests are dropped. This ensures smooth, predictable output rate regardless of bursty input.



Leaky Bucket: fixed queue, constant output rate · burst requests drop if queue full · smooth but inflexible

5. Fixed Window Counter

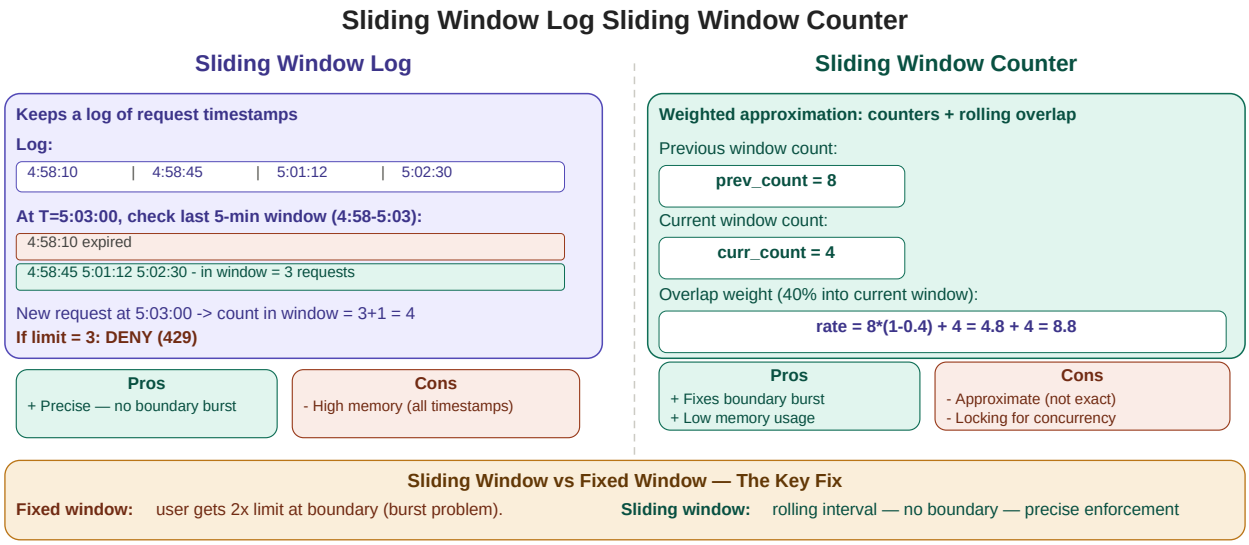
Divides time into fixed windows (e.g. every 5 minutes). Counts requests per user per window. Resets at each window boundary. Simple but suffers from a critical **boundary burst problem**.



Fixed window: simple counters per window · but boundary burst allows 2x the limit in a short period

6. Sliding Window Log & Sliding Window Counter

Both sliding window variants eliminate the boundary burst problem by using a **rolling time interval** instead of fixed resets. They differ in memory usage and accuracy.



Sliding Log: exact timestamps — precise but memory-heavy · Sliding Counter: weighted approximation — efficient and good enough

Property	Fixed Window	Sliding Window Log	Sliding Window Counter
Accuracy	Low (boundary burst)	Highest (exact)	Good (approx)
Memory	Very low (1 counter)	High (all timestamps)	Low (2 counters)
Complexity	Lowest	High	Medium
Burst handling	Double limit at edge	Precise prevention	Good approximation
Best for	Simple, non-critical	Security, finance	General production

7. Algorithm Comparison & Selection

Algorithm Selection Guide

Token Bucket	Leaky Bucket	Fixed Window	Sliding Window Log	Sliding Counter
Burst: YES	**Burst: NO (drops)**	**Burst: EDGE ISSUE**	**Burst: PRECISE**	**Burst: APPROX**
Memory: LOW	Memory: LOW	Memory: VERY LOW	Memory: HIGH	Memory: LOW
Accuracy: HIGH	Accuracy: HIGH	Accuracy: LOW	Accuracy: HIGHEST	Accuracy: GOOD
Complexity: LOW	Complexity: LOW	Complexity: LOWEST	Complexity: HIGH	Complexity: MED
Use: APIs, mobile apps	Use: streaming, constant rate	Use: simple APIs, low precision ok	Use: critical APIs finance, security	Use: distributed systems, Redis
MOST POPULAR	**STEADY RATE**	**SIMPLEST**	**MOST ACCURATE**	**BEST BALANCE**

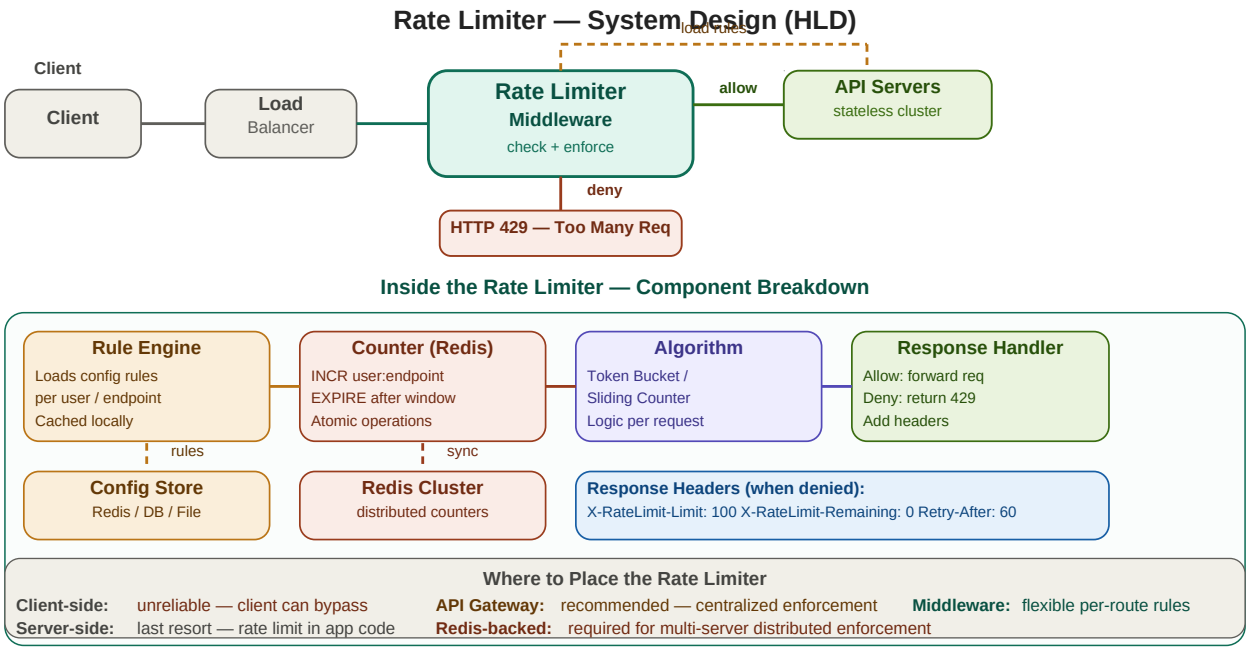
Quick Decision Guide

- **Need burst tolerance?** -> Token Bucket
- **Need constant rate?** -> Leaky Bucket
- **Need max precision?** -> Sliding Log
- **Low memory + precision?** -> Sliding Counter (Redis)
- **Simplest implementation?** -> Fixed Window (accept boundary flaw)

Five algorithms compared · Token Bucket = most popular · Sliding Counter = best balance for distributed systems

8. Rate Limiter — System Design (HLD)

A rate limiter is a **middleware layer** that intercepts every API request before it reaches your servers. It checks a shared counter store (Redis), applies the rate limiting algorithm, and either forwards the request or returns HTTP 429. Here is the full system design:



Rate Limiter HLD: Client -> LB -> Rate Limiter Middleware -> API Servers · Redis for distributed atomic counters · Config store for per-user/API rules

Key Design Decisions

Decision	Option A	Option B	Recommended
Where to place	Client-side (bypass risk)	API Gateway / Middleware	API Gateway
Counter storage	In-memory per server	Redis cluster (shared)	Redis cluster
Algorithm choice	Token Bucket (simple)	Sliding Counter (accurate)	Token Bucket for most
Atomicity	Application locks	Redis INCR (atomic)	Redis INCR + Lua
Config storage	Hardcoded	Config file / DB (hot reload)	Config DB cached

9. Implementation & Distributed Considerations

Single Server Implementation

On a single server, a hash map of {user_id: (counter, timestamp)} is sufficient. Update atomically using thread locks. Return 429 when limit exceeded.

Distributed Implementation

Multiple servers need a **shared counter store**. Each server cannot maintain its own counter — they would each allow the full limit, multiplying the effective rate.

Challenge	Solution
Multiple servers have separate counters	Use Redis as centralized counter store
Concurrent requests race condition	Redis INCR is atomic — no race condition
High latency from Redis on every req	Local cache + async sync (accept ~1% error)
Config changes need restart	Config in Redis/DB — hot reload without restart
Different limits per user/endpoint	Key = user_id:endpoint in Redis

Redis INCR + EXPIRE: INCR user:123:minute:{timestamp//60}, EXPIRE that key after 2 minutes. O(1) atomic counter.

Lua scripts in Redis: execute check + increment atomically — prevents race conditions without application-level locking.

At extreme scale: Sliding Window Counter in Redis with 2 keys (prev_window, curr_window) is the industry standard.

10. Quick Revision & Interview Questions

Quick Revision

-> Rate limiting prevents resource exhaustion from excessive requests. Returns HTTP 429 Too Many Requests.

-> Token Bucket: bucket holds N tokens, refills at fixed rate. Request consumes 1 token. Allows bursts. Most popular.

-> Leaky Bucket: fixed-size queue, processes at constant rate. Overflow dropped. No burst tolerance.

-> Fixed Window: counter per time window. Resets at boundary. Simple but allows 2x limit at window edges.

-> Fixed window boundary problem: 3 reqs at end of window A + 3 at start of window B = 6 in 1 second.

-> Sliding Window Log: stores all timestamps. Count in last T seconds. Most accurate but high memory.

-> Sliding Window Counter: weight prev + curr counters by overlap percentage. Good balance of accuracy and memory.

-> Distributed rate limiting: use Redis as shared counter. INCR is atomic. Key = user_id:window_timestamp.

-> Config-driven: rules per user, per API, per endpoint. Store in config file or Redis. Cache locally.

-> Race condition: two concurrent requests both read 0, both increment — use atomic operations or Lua scripts.

-> Token Bucket used by: AWS API Gateway, GitHub, Stripe, most major APIs.

-> No one-size-fits-all: choose based on burst tolerance, accuracy needs, memory constraints, complexity.

Interview Questions

- 1. What is rate limiting and why is it needed?
- 2. Explain the Token Bucket algorithm. How does it handle bursts?
- 3. What is the difference between Token Bucket and Leaky Bucket?
- 4. What is the Fixed Window Counter boundary burst problem? How do you fix it?
- 5. Explain Sliding Window Log. Why is it memory-intensive?
- 6. How does the Sliding Window Counter approximate the sliding window?
- 7. How would you implement rate limiting in a distributed system with 10 servers?
- 8. Why is Redis used for distributed rate limiting? What makes INCR special?
- 9. What HTTP status code is returned when rate limit is exceeded?
- 10. How would you allow different rate limits per API endpoint per user?
- 11. What are the trade-offs between accuracy and performance in rate limiting?

Key Terms

Rate Limiting	Restricting number of API calls within a time window to prevent abuse
HTTP 429	Too Many Requests — status code returned when rate limit exceeded
Token Bucket	Bucket holds N tokens, refills at rate R. Request consumes 1 token. Burst allowed.
Leaky Bucket	Fixed queue processed at constant rate. Overflow dropped. No burst tolerance.
Fixed Window	Counter per fixed time window, resets at boundary. Suffers boundary burst.
Boundary Burst	Fixed window flaw: user can send 2x limit by straddling two windows

Sliding Window Log	Store all request timestamps. Count in last T seconds. Most precise.
Sliding Window Counter	Weight prev+curr counters by time overlap. Good balance of accuracy/memory.
Redis INCR	Atomic increment in Redis — used for thread-safe distributed counters
Burst	Sudden spike of requests — Token Bucket allows it, Leaky Bucket drops it
Atomicity	Operation that completes fully or not at all — prevents race conditions
Config-driven	Rate limit rules from config/DB — allows per-user, per-endpoint limits